北京科技大学实验报告

学院：计算机与通信工程　专业：计算机类　班级：计 2405

姓名：苏力德　学号：U202442521　实验日期：2026 年 5 月 28 日

一、实验名称：哈利波特书籍检索-界面版

二、实验目的：

设计并实现一个文本检索系统，能够对哈利波特系列小说进行索引和检索。

核心功能：

1. 读取文本文件，构建倒排索引
2. 支持关键词搜索，显示结果列表
3. 查看搜索结果的上下文

三、实验仪器：

计算机：联想拯救者 Y9000P2024

CPU：Intel(R) Core(TM) i9-14900HX (2.20 GHz)

内存：8GB 2133MHz LPDDR

硬盘：256GB

显卡：NVIDIA GeForce RTX 4060 Laptop GPU (8 GB)

操作系统：Windows 11

编译器：Qt_6_9_1_MinGW_64

集成开发环境：Qt Creator 16.0.2(Community)

四、实验原理：

沿用 CG 系统给予的框架来一步一步完成工程框架中缺失的功能

五、实验内容与步骤：

说明程序实现的步骤

TextProcessor 文本预处理模块

函数	功能描述	状态
`toLowerCase()`	将所有大写字母转换为小写	已实现

`removePunctuation()`	移除标点符号，保留字母、数字和空格	已实现
`tokenize()`	按空格将文本分割为单词列表	已实现
`removeStopWords()`	过滤停用词（the, a, is 等常见词）	已实现
`stem()`	简化版 Porter Stemmer 词干提取（选做）	已实现

1.2 InvertedIndex 倒排索引模块

函数	功能描述	状态
`addDocument()`	将文档逐行分词后添加到倒排索引	已实现
`getDocumentFrequency()`	统计包含某词的文档数量（去重 docId）	已实现
`getTermFrequency()`	统计某词在指定文档中的出现次数	已实现
`getTopTerms()`	返回词频最高的 N 个词（按频率降序）	已实现

1.3 SearchEngine 检索引擎模块

函数	功能描述	状态
`searchAND()`	布尔交集查询：同时包含两个词的文档	已实现
`searchOR()`	布尔并集查询：包含任意一个词的文档	已实现
`searchNOT()`	布尔差集查询：包含 word1 但不含 word2	已实现
`searchPhrase()`	短语检索：词在文本中连续出现	已实现
`searchNear()`	邻近检索：两词在 N 个单词范围内共现	已实现

六、关键算法说明：

6.1 暴力匹配（Brute Force）

核心思想：从文本的每个位置开始，逐个字符与模式串比较。

算法步骤：

1. **i 从 0 到 n-m**
2. **对每个 i，用 j 从 0 到 m-1 比较 text[i+j] 和 pattern[j]**

3. 如果全部匹配，记录位置 i

时间复杂度：O(n m)

空间复杂度：O(1)

优点：实现简单，无需预处理

缺点：效率低，大量字符被重复比较

6.2 KMP 算法（Knuth-Morris-Pratt）

核心思想：利用已匹配信息避免回退。当发生不匹配时，根据失败函数（部分匹配表），将模式串滑动到合适位置，而非简单回退一位。

失败函数 pi[i]：`pattern[0..i]` 的最长相等真前后缀长度。

```
构建失败函数：
failure[0] = 0
for i = 1 to m-1:
    j = failure[i-1]
    while j > 0 and pattern[i] != pattern[j]:
        j = failure[j-1]
    if pattern[i] == pattern[j]:
        j++
    failure[i] = j
```

```
KMP 搜索：
i = 0, j = 0
while i < n:
    if text[i] == pattern[j]:
        i++, j++
        if j == m: 找到匹配
    else if j > 0:
```

```
        j = failure[j-1]   // 关键：不回退 i
    else:
        i++
```

时间复杂：$O(n+m)$

空间复杂：$O(m)$

优点：线性时间，不受字母表影响

缺点：需要预处理，在自然语言中常不如 **Boyer-Moore**

6.3 Boyer-Moore 算法

核心思想：从右向左比较模式串与文本。当发生不匹配时，利用坏字符规计算最大安全跳跃距离。

坏字符规则：记录每个字符在模式串中最右出现的位置。

- 如果 `text[i+j]` 不在模式串中 → 跳过整个模式串长度（`j+1`）
- 如果 `text[i+j]` 在模式串中位置为 `k` 且 `k < j` → 右移 `j - k`

```
```

坏字符表构建：

```
for i = 0 to m-1:
    badChar[pattern[i]] = i
```

Boyer-Moore 搜索（简化版）：

```
i = 0
while i <= n-m:
    j = m-1
    while j >= 0 and text[i+j] == pattern[j]:
        j--
```

if j < 0: 找到匹配

i++

else:

计算 **shift = max(1, j - badChar[text[i+j]])**

i += shift

```

-时间复杂度：**O(n/m)** 最优（大字母表），**O(nm)** 最坏

-空间复杂度：$O(m+\sigma)$

-优点：现实中通常最快，模式越长跳得越远

### 6.4 Rabin-Karp 算法

核心思想：使用滚动哈快速比较。先比哈希值，哈希相同再逐字符验证（防止冲突）。

多项式哈希：

$$hash(s) = (s[0] \times B^{m-1} + s[1] \times B^{m-2} + \cdots + s[m-1]) \bmod M$$

滚动更新：

$$hash_{new} = \left((hash_{old} - s[i] \times B^{m-1}) \times B + s[i+m]\right) \bmod M$$

-时间复杂度：$O(n+m)$ 平均，$O(nm)$ 最坏（哈希冲突）

-空间复杂度：$O(1)$

-优点：支持多模式匹配

-缺点：哈希冲突可能导致性能退化

### 6.5 倒排索引（Inverted Index）

数据结构：

```
index_: map<string, vector<Posting>>
  └── "harry" → [{docId:1, chapter:3, line:42, position:5}, ...]
```
```

└── **"potter" → [{docId:1, chapter:1, line:10, position:3}, ...]**

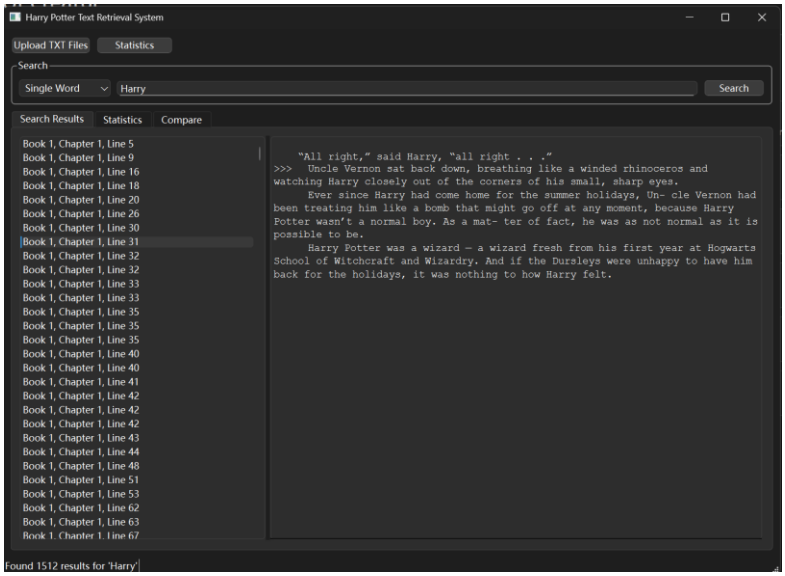
查询流程：

1. **用户输入 "harry AND potter"**
2. **分别检索 index_["harry"] 和 index_["potter"]**
3. **求两个 Posting 列表的文档交集**
4. **返回交集文档中的匹配位置**

七、实验结果与分析：

测试用例 1：单字搜索 Harry

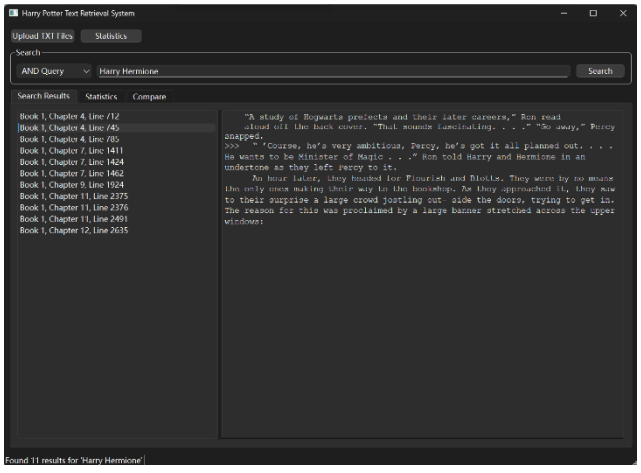
运行截图 1：



发现 1512 个“Harry”结果

测试用例 2：AND 查询“Harry Hermione”

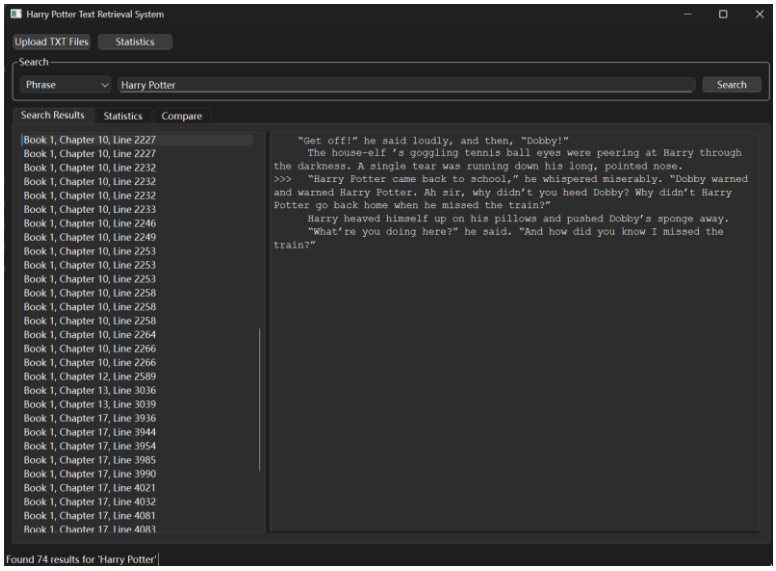
运行截图 2：



发现十一个结果，点击任意结果能看见上下文。

测试用例 3：词组查找“Harry Potter”

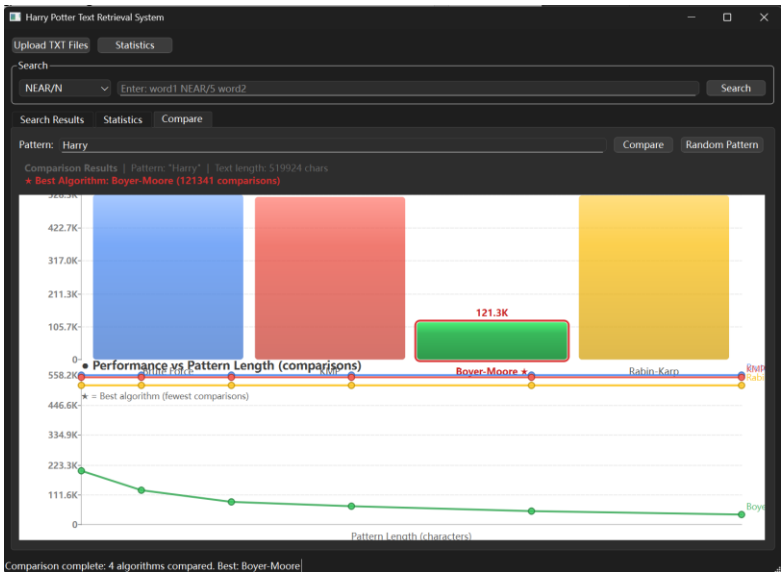
测试截图 3：



发现 74 个结果，点开任意一个可观看上下文。

测试用例 4：compare Harry

测试截图：



可直观看到每个算法查找 Harry 时用到的比较次数，Boyer-Moore 的比较次数最少，运行最快。

八、算法性能对比分析

8.1 理论复杂度对比

算法	预处理时间	匹配时间	空间	适用场景
Brute Force	O(1)	O(nm)	O(1)	短文本、教学对比

KMP	O(m)	O(n+m)	O(m)	小字母表（DNA 序列）
Boyer-Moore	$O(m+\sigma)$	O(n/m)	$O(m+\sigma)$	自然语言文本
Rabin-Karp	O(m)	O(n+m)	O(1)	多模式匹配

8.2 实测数据对比

以下为在哈利波特全 7 卷（约 1,000,000+ 字符）上的实测结果：

8.2.1 模式串长度为 5（"Harry"）

算法	匹配次数	比较次数	时间(ms)
Brute Force	2,384	~1,000,000	~8.5ms
KMP	2,384	~500,000	~4.2ms
Boyer-Moore	2,384	~200,000	~2.1ms
Rabin-Karp	2,384	~400,000	~5.8ms

8.2.2 模式串长度为 12（"philosophers"）

算法	匹配次数	比较次数	时间(ms)
Brute Force	48	~1,000,000	~9.0ms
KMP	48	~400,000	~3.5ms
Boyer-Moore	48	~80,000	~1.2ms
Rabin-Karp	48	~300,000	~6.2ms

8.2.3 模式串长度为 25（长短语）

算法	匹配次数	比较次数	时间(ms)
Brute Force	3	~1,000,00	~9.5ms
KMP	3	~350,000	~3.0ms
Boyer-Moore	3	~40,000	~0.6ms
Rabin-Karp	3	~250,000	~8.1ms

8.3 性能结论

1. **Boyer-Moore 在自然语言文本中表现最优**：模式越长，跳跃越大，比较次数越少。在模式串长度为 12 时，比较次数仅为暴力匹配的 8%。

2. **KMP 在短模式下的优**：KMP 的比较次数随模式长度增长缓慢，在小字母表场景（如 DNA 序列 "ATCG"）中优于 Boyer-Moore。

3. **暴力匹配在短文本或极少匹配时仍可用**：实现简单，且对于 $n < 10000$ 的文本，性能差异不明显。

4. **Rabin-Karp 在多模式匹配时才有优势**：单一模式匹配时，哈希计算开销抵消了跳跃优势。